Evaluation Metrics

Accuracy

Definition. The ***accuracy*** of a classifier $\boldsymbol{M}$ on a dataset of n examples is given by:

$$Accuracy(\boldsymbol{M}) = \frac{|\{i : \boldsymbol{M}(\mathbf{x}_i) = y_i\}|}{n} = \frac{\#\text{ correct predictions}}{n}$$

where:

- $\mathbf{x}_i$: input features for example i,
- $y_i \in \{1, 2, \ldots, k\}$: example i is one of k classes, and
- $\boldsymbol{M}(\mathbf{x}_i) \in \{1, 2, \ldots, k\}$: predicted label for example i.

Example. Suppose we have an animal image classifier with $k = 3$ classes: $\{cat, bird, dog\}$.

Also suppose we have a dataset with 100 images and the following results.

$$Accuracy(\boldsymbol{M}) = \frac{70 + 12 + 2}{100} = 0.84$$

	Predicted Cat	Predicted Dog	Predicted Bird	TOTAL
Actual Cat	**70**	5	0	75
Actual Dog	8	**12**	0	20
Actual Bird	3	0	**2**	5
TOTAL	81	17	2	100

What is the problem with using accuracy with this dataset?

A naive classifier always predicting "cat" gets 75% accuracy.

High overall accuracy masks poor performance on rare classes!

The Confusion Matrix

We observed the use of a matrix to present the results of a classifier in the last example. Let's formalize this notion.

Definition. For a classifier with $k > 2$ classes, the ***confusion matrix*** is a $k \times k$ table given by:

Predicted / Actual	$\hat{y} = 1$	$\hat{y} = 2$	…	$\hat{y} = k$
$y = 1$	$C_{1,1}$	$C_{1,2}$	…	$C_{1,k}$
$y = 2$	$C_{2,1}$	$C_{2,2}$	…	$C_{2,k}$
…	…	…	…	…
$y = k$	$C_{k,1}$	$C_{k,2}$	…	$C_{k,k}$

where $C_{i,j}$ is the count: $|\{examples\ with\ true\ class\ i, predicted\ as\ class\ j\}|$.

The main diagonal gives us the counts of correct predictions; all others are mis-classifications.

The Confusion Matrix

It is most common for us to consider confusion matrices for binary classifiers and refer to the labels as $\{positive, negative\}$.

There are 4 possible outcomes we visualize as the ***binary confusion matrix***.

	Predicted Positive	Predicted Negative
Actually Positive	True Positive (TP)	**False Negative (FN)**
Actually Negative	**False Positive (FP)**	True Negative (TN)

When $k > 2$, per-class metrics are obtained by reducing to k binary classification problems (one-vs-all).

In the $\{cat, bird, dog\}$ example, we would treat cats as positive, {dog, bird} as negative, then repeat for each class taking on the positive moniker.

Evaluating Impact vs. Accuracy I

In many real-world scenarios, one type of error is much worse than another.

Consider a scenario where missing a disease diagnosis is dangerous.

Actual \ Predicted	Sick	Healthy
Sick	**CORRECT DIAGNOSIS** (TP) Patient receives life-saving treatment. Outcome: **Survival**	**MISSED DIAGNOSIS** (FN) Patient is sent home while sick; disease progresses without care. Outcome: **Dangerous / Critical**
Healthy	**FALSE ALARM** (FP) Patient undergoes unnecessary stress and extra testing. Outcome: **Inconvenience / Anxiety**	**CLEAN BILL OF HEALTH** (TN) Healthy patient is correctly reassured. Outcome: **Peace of Mind**

Evaluating Impact vs. Accuracy II

Consider a scenario where the justice system must decide if a defendant is guilty or innocent.

Actual \ Predicted	Guilty	Not Guilty
Guilty	**JUSTICE SERVED** (TP) The perpetrator is held accountable for their actions. Outcome: **Justice**	**ACQUITTAL BY ERROR** (FN) A guilty person is released back into society. Outcome: **Public Risk**
Not Guilty	**WRONGFUL CONVICTION** (FP) An innocent person is convicted and deprived of their liberty. Outcome: **Miscarriage of Justice**	**EXONERATION** (TN) The innocent person is correctly cleared of all charges. Outcome: **Freedom Preserved**

Questions

Actual \ Predicted	Positive	Negative
Positive	True Positive (TP)	**False Negative (FN)**
Negative	**False Positive (FP)**	True Negative (TN)

If you were designing an Airport Security Scanner, which box would you be most afraid of, and why?

False Negative ("Miss"): A threat (weapon/explosive) passes through undetected, risking catastrophic loss of life.

False Positive ("False Alarm"): A harmless item triggers an alarm, resulting only in a minor delay or manual search.

A **False Positive** means you just blocked a loyal customer's legitimate purchase. The Consequence may be that the immediate sale is lost, we frustrate the customer, and likely drive them to a competitor.

(A **False Negative** occurs when a stolen credit card is used, but the model fails to flag it and allows the transaction to go through.)

If you are an e-commerce giant building a "Fraud Detection" system to auto-block credit cards, why are you terrified of a False Positive?

When an automated sensor scans $1,000 smartphone screens for cracks, why is a mistake in either direction a disaster?

Both mistakes hit the "bottom line" equally hard.

- The **False Negative** is a cracked screen reaches a customer, leading to expensive returns and a damaged reputation.
- The **False Positive** is a perfect, expensive screen is thrown in the trash, directly wasting profit.

Precision

If your email spam filter identifies 100 emails as ***Junk***, but 20 of them were actually important messages, is your filter doing a good job?

The Experience: The user stops trusting the filter. They feel forced to check the *Junk* folder every day anyway, just in case the filter made a mistake.

The Lesson: It is not enough to catch a lot of junk; you have to be accurate in your accusations. If you say something is *Junk*, you better be right, or you are just creating more work for the user.

We need a way to measure ***trust***.

We need a measurement that tells us: Of all the times the model predicted *positive*, how often was it actually right?

Definition. The ***Precision*** of a binary classifier M is the fraction of *positive* predictions that are correct:

$$Precision(M) = \frac{TP}{TP + \boldsymbol{FP}}$$

Actual \ Predicted	Positive	Negative
Positive	TP	$\boldsymbol{FN}$
Negative	$\boldsymbol{FP}$	TN

Recall

If a massive earthquake is starting, but your system stays silent because it wasn't "100% sure" it wasn't just a heavy truck driving by, what happens?

The Experience: You do not care if the alarm goes off once in a while for a false vibration (a False Alarm). You only care that when a real disaster happens, the alarm sounds every single time.

The Lesson: When lives are at stake, hesitation is the greatest risk. We prefer a system that is highly sensitive: one that triggers easily rather than one that waits for perfect certainty while the danger slips by.

We need a way to measure ***coverage***.

We need a measurement that tells us: Of all the actual *positive* cases that exist, how many did the model find?

Definition. The ***Recall*** of a binary classifier M is the fraction of actual *positives* that are correctly identified:

$$Recall(M) = \frac{TP}{TP + \boldsymbol{FN}}$$

Actual \ Predicted	Positive	Negative
Positive	TP	$\boldsymbol{FN}$
Negative	$\boldsymbol{FP}$	TN

Recall **is finding every rock that falls; in safety, we maximize this metric to ensure a single miss does not become a catastrophe.**

Precision & Recall ↔ Soundness & Completeness

A reminder about what we learned from Logic. A proof system or reasoning system is:

Sound: Every conclusion is correct.

No false positives!

Complete Every truth can be proven.

No false negatives!

Logic	Classification	Metric
Sound (no false proofs)	Few false positives	$Precision$
Complete (proves all truths)	Few false negatives	$Recall$
Sound and Complete (Both approaches always give the same answer.)	Perfect Positive Classification { Predicted positive } = { Actual positive }	$Precision = Recall = 1$

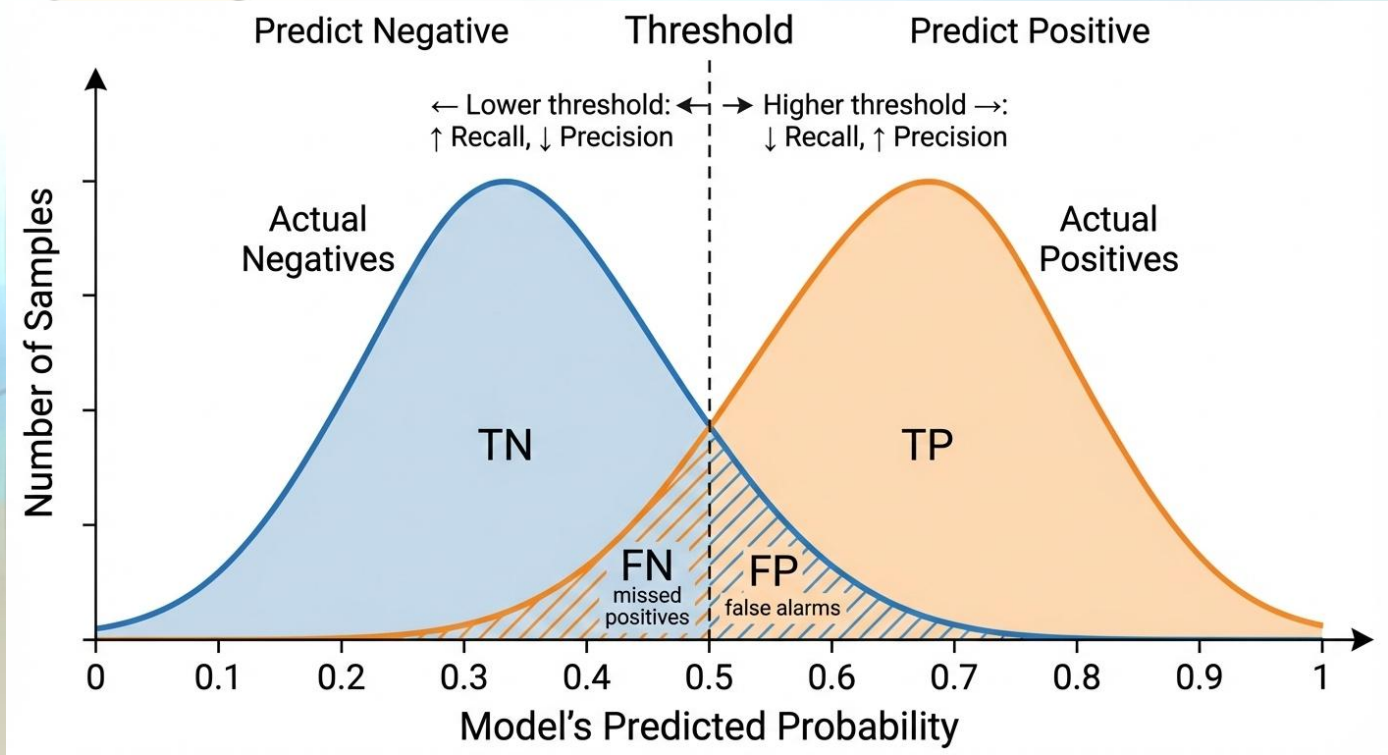
Important Note: This only guarantees correct handling of positives. (True negatives are not considered.)

In logic, there are many interesting systems that are either sound OR complete, not both. In Machine Learning, most classifiers must trade off $Precision$ and $Recall$; achieving both equal to 1 is rare.

The Precision-Recall Tradeoff

$$Precision(M) = \frac{TP}{TP + \boldsymbol{FP}}$$

$$Recall(M) = \frac{TP}{TP + \boldsymbol{FN}}$$



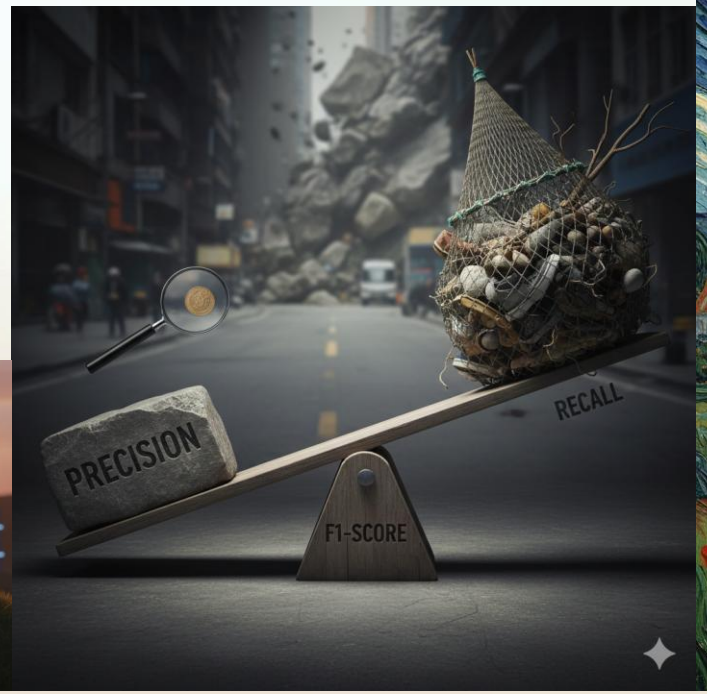
- **Blue curve**: Distribution of actual negatives across predicted probabilities.
- **Orange curve**: Distribution of actual positives across predicted probabilities
- **Overlap region**: Where the model is uncertain: where errors happen

The Tradeoff

- **Move threshold left**: Catch more positives (↑ Recall) but more false alarms (↓ Precision)
- **Move threshold right**: Fewer false alarms (↑ Precision) but miss more positives (↓ Recall)

There is no "correct" threshold — it depends on the cost of each type of error.

With a measurement, how can we balance $Precision$ and $Recall$?



Harmonic Mean

Calculate the average speed a car travels on a round trip where outbound we travel 60 miles at 30 mph and on the return trip, we travel 60 miles at 60 mph.

We can informally compute the ***total time*** and then the ***average speed***.

$$Total\ time = \frac{60miles}{30mph} + \frac{60miles}{60mph} = 3hours \qquad Avg\ speed = \frac{distance}{time} = \frac{120miles}{3\ hours} = 40mph$$

Let us try different averaging mechanisms.

	Arithmetic Mean	**Geometric Mean**	**Harmonic Mean**
Average	$\frac{30+60}{2} = 45\ mph$	$\sqrt{30 \cdot 60} = 42.426\ mph$	$\frac{2}{\left(\frac{1}{30}+\frac{1}{60}\right)} = 40.0\ mph$
Predicted Total Time	$\frac{1}{45}\ hpm \cdot 120\ miles = 2.67\ hours$	$\frac{1}{42.426}\ hpm \cdot 120\ miles = 2.828\ hours$	$\frac{1}{40.0}\ hpm \cdot 120\ miles = 3.0\ hours$

Speed is a ***rate*** $\left(\frac{distance}{time}\right)$. When averaging rates, we use the harmonic mean.

$$\boxed{H = \frac{n}{\frac{1}{r_1} + \frac{1}{r_2} + \cdots \frac{1}{r_n}}}$$

F_1 Score

$$H = \frac{n}{\frac{1}{r_1} + \frac{1}{r_2} + \cdots \frac{1}{r_n}}$$

How can we average $Precision$ (P) and $Recall$ (R)?

Note, each of these values are rates (with no units).

So, $H = \dfrac{2}{\frac{1}{P} + \frac{1}{R}} = \dfrac{2}{\frac{P+R}{PR}} = \dfrac{2PR}{P+R}$

Definition. The $\boldsymbol{F_1}$ ***score*** of a binary classifier M is the harmonic mean of precision and recall:

$$F_1(M) = \frac{2 \cdot Precision(M) \cdot Recall(M)}{Precision(M) + Recall(M)}$$

where $Precision(M) = \frac{TP}{TP+\boldsymbol{FP}}$ and $Recall(M) = \frac{TP}{TP+\boldsymbol{FN}}$.

$$F_\beta(M) = (1+\beta^2)\frac{Precision(M) \cdot Recall(M)}{\beta^2 \cdot Precision(M) + Recall(M)}$$

We can also give weight to $Precision$ or $Recall$ using the generalized β version.

When $\beta = 1$ each have equal weight as defined above.

$\beta = 2$ weights $Recall$ twice as much as $Precision$ (since β^2 is in the denominator).

Examples: F_1 Score

Calculate F_1 for each confusion matrix.

Actual \ Predicted	Spam	Not
Spam	47	3
Not	47	3

Actual \ Predicted	Spam	Not
Spam	25	25
Not	1	49

Actual \ Predicted	Spam	Not
Spam	45	5
Not	5	45

Precision

$$\frac{47}{47+47} = 0.5$$

$$\frac{25}{25+1} = 0.9615$$

$$\frac{45}{45+5} = 0.9$$

Recall

$$\frac{47}{47+3} = 0.94$$

$$\frac{25}{25+25} = 0.5$$

$$\frac{45}{45+5} = 0.9$$

$\boldsymbol{F_1}$

$$\frac{2 \cdot 0.5 \cdot 0.94}{0.5+0.94} = 0.6528$$

$$\frac{2 \cdot 0.9615 \cdot 0.5}{0.9615+0.5} = 0.6579$$

$$\frac{2 \cdot 0.9 \cdot 0.9}{0.9+0.9} = 0.9$$

Too many false alarms.

Misses too much spam.

'Well' balanced.

Remember: Cross Entropy as an Evaluation Metric

From our NLP discussion on entropy and information theory... **Cross Entropy** measures how well a model's predicted distribution matches the true distribution.

Definition. For two probability distributions $\mathbb{T}$ (ground truth distribution) and $\boldsymbol{M}$ (model distribution) over the same set of outcomes $\{c_1, c_2, \dots, c_K\}$, the ***cross entropy*** of $\mathbb{T}$ with respect to $\boldsymbol{M}$ is given by

$$CrossEntropy(\mathbb{T}, \boldsymbol{M}) = -\sum_{i=1}^{K} p_{c_i} \log_2 q_{c_i}$$

where p_{c_i} is the true probability of outcome c_i and q_{c_i} is the predicted probability of outcome c_i.

Recall: Cross entropy measures how "surprised" a model is by the actual outcomes.

- Low CE → The model expected what actually happened
- High CE → The model was surprised by reality

- We sample from $\mathbb{T}$ (reality)
- We measure surprise using $\boldsymbol{M}$'s beliefs
- $\log_2 q_{c_i}$ is the model's surprise when event c_i occurs
- We weight by p_{c_i} how often c_i actually happens

Why Confidence Matters

Suppose two spam classifiers both achieve 80% accuracy. Are they equally good?

	Model A (Calibrated)	Model B (Overconfident)
When correct	“Probably spam”	“Definitely spam”
When wrong	“Not sure, maybe spam?”	“Definitely spam”
Cross Entropy	**Lower**	**Higher**

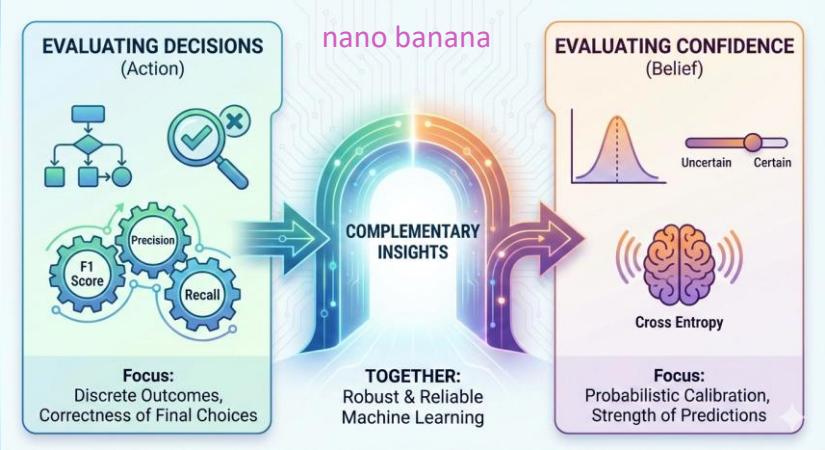
Why the difference?

- Model B is confidently wrong with its mistakes.
- Cross entropy penalizes confident mistakes more heavily than hesitant ones.

Accuracy tells you *what* the model got right; however, ***cross entropy*** tells you *how well the model's probability estimates match reality.*

In some cases, cross entropy reveals problems that accuracy hides.

- Medical diagnosis: “90% vs 55% chance of cancer” require different follow-ups, even if both exceed a threshold
- Weather forecasting: “90% vs 60% chance of rain” affects whether you cancel the outdoor wedding.
- ER triage: Must prioritize patients by actual risk level, not just “at risk” vs “not at risk”



Cross Entropy vs. Accuracy/F_1/Precision/Recall

These metrics answer different questions.

	Accuracy / F1 / Precision / Recall	Cross Entropy
Question	"Did we decide correctly?"	"Are our probabilities trustworthy?"
Operates on	Hard predictions (class labels)	Soft predictions (probabilities)
Evaluates	Decision quality	Probability quality
Throws away	Probability information	Nothing

Suppose we have predictions with probabilities

$$p = 0.51 \text{ and } p = 0.99.$$

These are *identical* to accuracy/F_1/precision/recall (both predict positive classification), but they are *very different* to cross entropy.

These metrics are complementary, not competing: use accuracy/F_1/precision/recall to evaluate decisions, use cross entropy to evaluate confidence.

Beyond F_1: True Negatives

Actual \ Predicted	Positive	Negative
Positive	TP	$\boldsymbol{FN}$
Negative	$\boldsymbol{FP}$	TN

F_1 Score combines Precision and Recall, but ignores True Negatives.

This matters as many applications, correctly identifying negatives is just as important as identifying positives.

If you were designing an Airport Security Scanner, which box would you be most afraid of, and why?

Classifier A

	Predicted Threat	Predicted Safe
Actual Threat	40	60
Actual Safe	200	700

$Precision = \frac{40}{240} = 0.167$ $Recall = \frac{40}{100} = 0.4$ $F1 = 0.235$

Classifier B

	Predicted Threat	Predicted Safe
Actual Threat	35	65
Actual Safe	10	890

$Precision = \frac{35}{45} = 0.777$ $Recall = \frac{35}{100} = 0.35$ $F1 = 0.4828$

- Classifier A flags 200 innocent passengers compared to 10 for Classifier B
- 200 more false positives means 200 fewer true negatives.

The false positives are a critical operational difference; the F_1 scores slightly reveal this cost.

How We Got Here

Metric	What It Tells Us	What It Ignores	Consequence	
Accuracy	Overall correctness: percentage of all predictions that are correct	Class distribution	With imbalanced datasets, a naive "always predict majority class" classifier can achieve high accuracy	$\frac{TP + TN}{TP + TN + FN + FP}$
Precision	Trust in positive predictions: when we predict positive, how often are we right?	False negatives	Can achieve perfect precision by being extremely conservative and missing most positives	$\frac{TP}{TP + FP}$
Recall	Coverage of positives: of all actual positives, how many did we find?	False positives	Can achieve perfect recall by predicting everything as positive	$\frac{TP}{TP + FN}$
F_1	Balance between precision and recall for the positive class	True negatives	Does not account for the cost of false positives when they create operational burdens	$F_1(M) = \frac{Precision(M) \cdot Recall(M)}{Precision(M) + Recall(M)}$
MCC	Overall correlation between predictions and reality across all four confusion matrix cells	Different error costs	Treats all errors equally; may not reflect scenarios where one error type is much more costly	

Toward Matthews Correlation Coefficient (MCC)

Actual \ Predicted	Positive	Negative
Positive	TP	$\boldsymbol{FN}$
Negative	$\boldsymbol{FP}$	TN

What would a truly balanced metric look like?

We need a metric that:

- Considers all four confusion matrix cells (not just TP, $\boldsymbol{FP}$, $\boldsymbol{FN}$)
- Measures overall agreement between predictions and reality
- Treats both classes symmetrically

Can we measure agreement? Disagreement?

$$TP \cdot TN \qquad \boldsymbol{FP} \cdot \boldsymbol{FN}$$

[Multiplication enforces an 'and' (joint) condition with the components.]

We can thus compute the difference between how much we (dis)agree:

$$TP \cdot TN - \boldsymbol{FP} \cdot \boldsymbol{FN}$$

Then, we need to normalize.

Matthews Correlation Coefficient (MCC)

MCC is a balanced correlation between predictions and truth, giving equal weight to all four parts of the confusion matrix.

Actual \ Predicted	Positive	Negative
Positive	TP	$\boldsymbol{FN}$
Negative	$\boldsymbol{FP}$	TN

$$MCC = \frac{TP \cdot TN - \boldsymbol{FP} \cdot \boldsymbol{FN}}{\sqrt{(TP + \boldsymbol{FP})(TP + \boldsymbol{FN})(TN + \boldsymbol{FP})(TN + \boldsymbol{FN})}}$$

Predicted Positive | Actual Positives | Actual Negatives | Predicted Negatives

Agreement - Disagreement

: marginals

Recall the correlation coefficient r from statistics? That is the ***Pearson Correlation Coefficient*** (***PCC***): a measure of how two variables move together.

MCC applies the same correlation idea to binary outcomes.

Value	Meaning
+1	Perfect prediction
0	No better than random guessing
-1	Total disagreement (perfectly wrong)

MCC Example

If you were designing an Airport Security Scanner, which box would you be most afraid of, and why?

Classifier A

	Predicted Threat	Predicted Safe
Actual Threat	40	60
Actual Safe	200	700

Classifier B

	Predicted Threat	Predicted Safe
Actual Threat	35	65
Actual Safe	10	890

$$MCC = \frac{40 \cdot 700 - \mathbf{200} \cdot \mathbf{60}}{\sqrt{(40 + \mathbf{200})(40 + \mathbf{60})(700 + \mathbf{200})(700 + \mathbf{60})}} = 0.1249$$

$$MCC = \frac{35 \cdot 890 - \mathbf{10} \cdot \mathbf{65}}{\sqrt{(35 + \mathbf{10})(35 + \mathbf{65})(890 + \mathbf{10})(890 + \mathbf{65})}} = 0.4904$$

MCC clearly distinguishes the better classifier with greater granularity than F_1 (0.235 vs 0.4828).

MCC vs. F_1: A Direct Comparison

Suppose we have three spam filters, each with 50 spam and 50 legitimate emails:

	Pred Spam	Pred Not
Actual Spam	47	3
Actual Not	3	47
Actual Spam	25	25
Actual Not	1	49
Actual Spam	48	2
Actual Not	48	2

Precision	Recall	F1	MCC
94%	94%	94%	0.88
96%	50%	66%	0.53
50%	96%	66%	0.53

- Filters B and C have identical F1 scores but very different behavior.
- MCC correctly identifies both as equally mediocre.

Best practice is to report both metrics: F_1 for alignment with the existing literature, and MCC to capture truly balanced performance.

MCC is Balanced

$$MCC = \frac{TP \cdot TN - \boldsymbol{FP} \cdot \boldsymbol{FN}}{\sqrt{(TP + \boldsymbol{FP})(TP + \boldsymbol{FN})(TN + \boldsymbol{FP})(TN + \boldsymbol{FN})}}$$

With MCC, all four cells of the confusion matrix contribute equally to the result. This symmetry makes MCC particularly reliable for:

- Imbalanced datasets
- Cases where both classes matter equally
- Comparing classifiers with different precision/recall tradeoffs

Choosing Your Metric

	Accuracy	Precision	Recall	F1	MCC
Handles class imbalance?	No	Partial	Partial	Partial	Yes
Considers TN?	Yes	No	No	No	Yes
Needs Companion Metrics?	Yes	Yes	Yes	Yes	No
Fully balanced?	No	No	No	Partial	**Yes**
Best for…	Balanced data	Trust	Coverage	Positive focus	**Overall balance**